BMS COLLEGE OF ENGINEERING

(Autonomous college, affiliated to Visveswaraya Technological University)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION

V Semester

ADVANCED DIGITAL LOGIC DESIGN

(23EC5PE1AD)

AAT Report

Topic: Digital Logic Modules

Sl No	Student Name	USN	Student signature	Signature of the Staff In charge	Marks Obtained
1.	Venkatesh	1BF24EC185			

Course Instructor:

Prof. Ajaykumar Devarapalli
Dept. of ECE, BMSCE

Table of Contents

1. Arithmetic Logic Unit (ALU)
2. Comparator
3. 4-Bit Parity Generator and Checker
4. 4-Bit Barrel Shifter
5. Timer Module
6. Ring Counter
7. Johnson Counter
8. Synchronous and Asynchronous Counters
9. Shift Register
10. Divide-by-N Counter (Using Modulo-6)
11. Mealy and Moore FSMs
12. Random Sequence Generator (LFSR)

Tools Used

The design, implementation, and simulation of all 12 digital logic modules in this report were carried out using standard Hardware Description Language (HDL) techniques and browser-accessible Electronic Design Automation (EDA) simulation platforms. This section outlines the software suite and toolchain utilized for modern digital design verification.

1. Verilog HDL

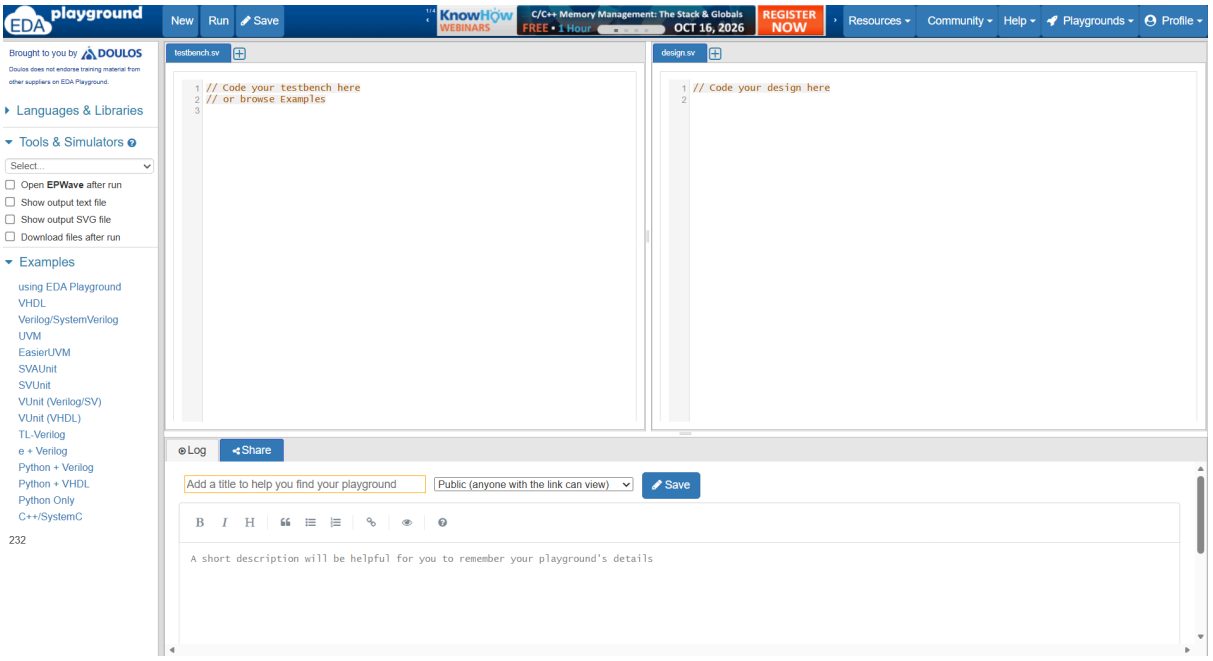
Verilog Hardware Description Language (IEEE 1364 standard) was selected for behavioral, dataflow, and structural modeling of all digital sub-systems. Its concise syntax and widespread industry adoption make it ideal for modeling both combinational logic (such as multiplexers, parity checkers, and ALUs) and sequential systems (such as state machines, shift registers, and counters).

2. Icarus Verilog Simulator

Icarus Verilog (iVerilog) served as the primary open-source Verilog simulation and synthesis engine. The tool compiles the Verilog HDL source files alongside testbench modules to evaluate behavioral correctness and generate Value Change Dump (VCD) trace files for subsequent waveform analysis.

3. EPWave via EDA Playground

EDA Playground provided an integrated cloud-based environment to host the iVerilog toolchain. Output VCD files generated during simulation runs were rendered visually using EPWave (Electronic Playground Waveform Viewer). This allowed for precise verification of signal transitions, clock cycles, propagation delays, and logic output validity across all test cases.



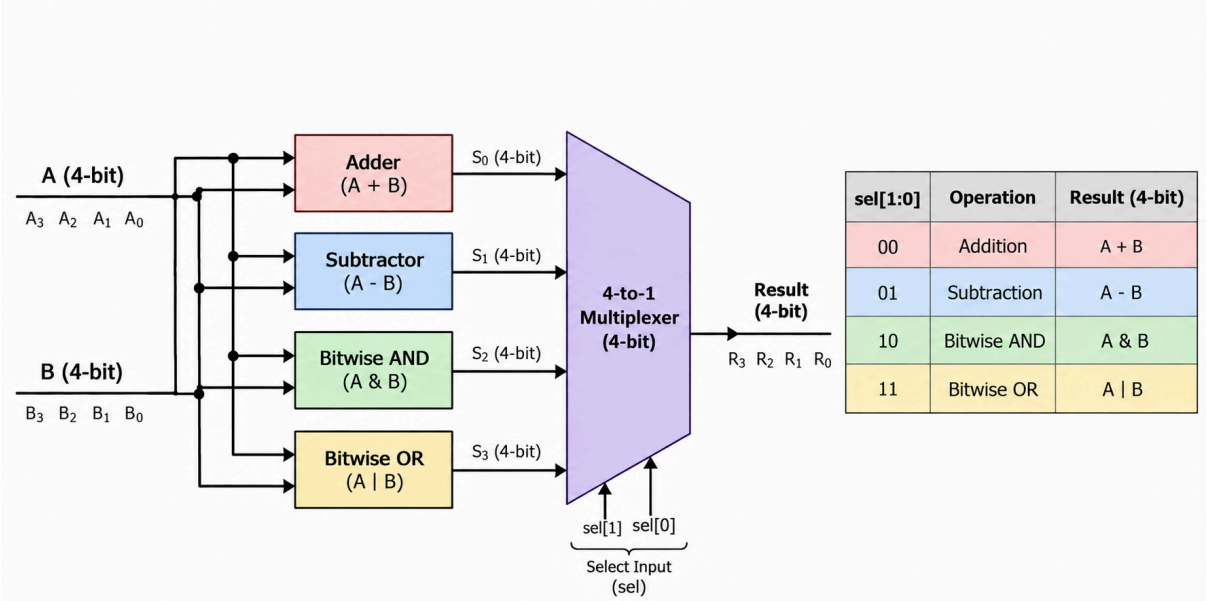
1. Arithmetic Logic Unit (ALU)

Design

The 4-Bit ALU uses a select input (`sel`) to determine which operation to perform on the 4-bit inputs `A` and `B`. It uses combinational logic represented by multiplexers and arithmetic/logical operators.

- `00`: Addition (`A + B`)
- `01`: Subtraction (`A - B`)
- `10`: Bitwise AND (`A & B`)
- `11`: Bitwise OR (`A | B`)

Circuit Diagram & Truth Table



sel[1:0]	Operation	Result (4-bit)
00	Addition	A + B
01	Subtraction	A - B
10	Bitwise AND	A & B
11	Bitwise OR	A | B

Figure 1.1: 4-Bit ALU Block Diagram and Operation Truth Table

Verilog HDL Code

```
None
module alu_4bit (input [3:0] a, b, input [1:0] sel, output reg [3:0] result);
  always @(*) begin
    case(sel)
      2'b00: result = a + b;
      2'b01: result = a - b;
      2'b10: result = a & b;
      2'b11: result = a | b;
```

```
    endcase
  end
endmodule
```

Testbench

None

```
module tb_alu_4bit;
  reg [3:0] a, b; reg [1:0] sel; wire [3:0] result;
  alu_4bit uut (a, b, sel, result);
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_alu_4bit);
    a = 4'b1010; b = 4'b0011;
    sel = 2'b00; #10;
    sel = 2'b01; #10;
    sel = 2'b10; #10;
    sel = 2'b11; #10;
  end
endmodule
```

Output

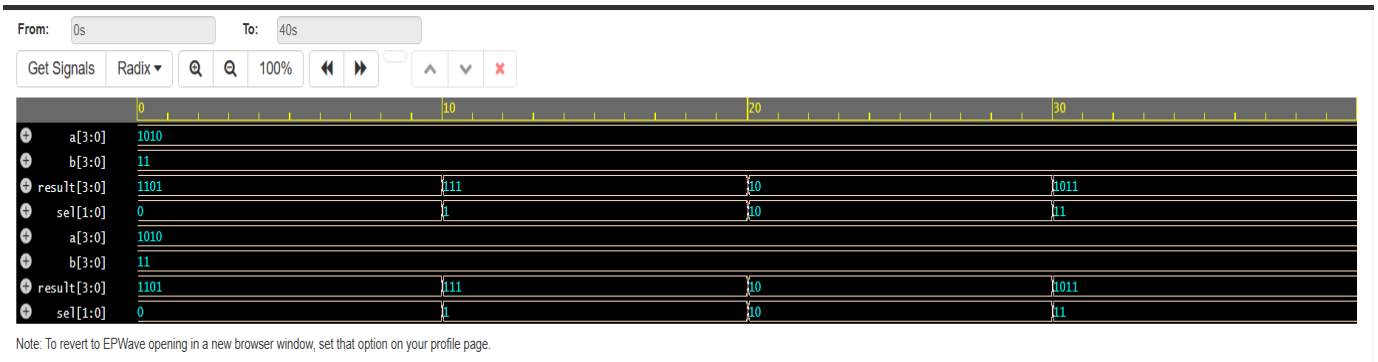


Figure 1.2: 4-Bit ALU Simulation Waveform

2. Comparator

Design

Compares two 4-bit binary values `A` and `B` to decide if `A` is greater, equal, or less than `B`.

- `a_gt_b` is high if `A > B`
- `a_eq_b` is high if `A == B`
- `a_lt_b` is high if `A < B`

Circuit Diagram & Truth Table

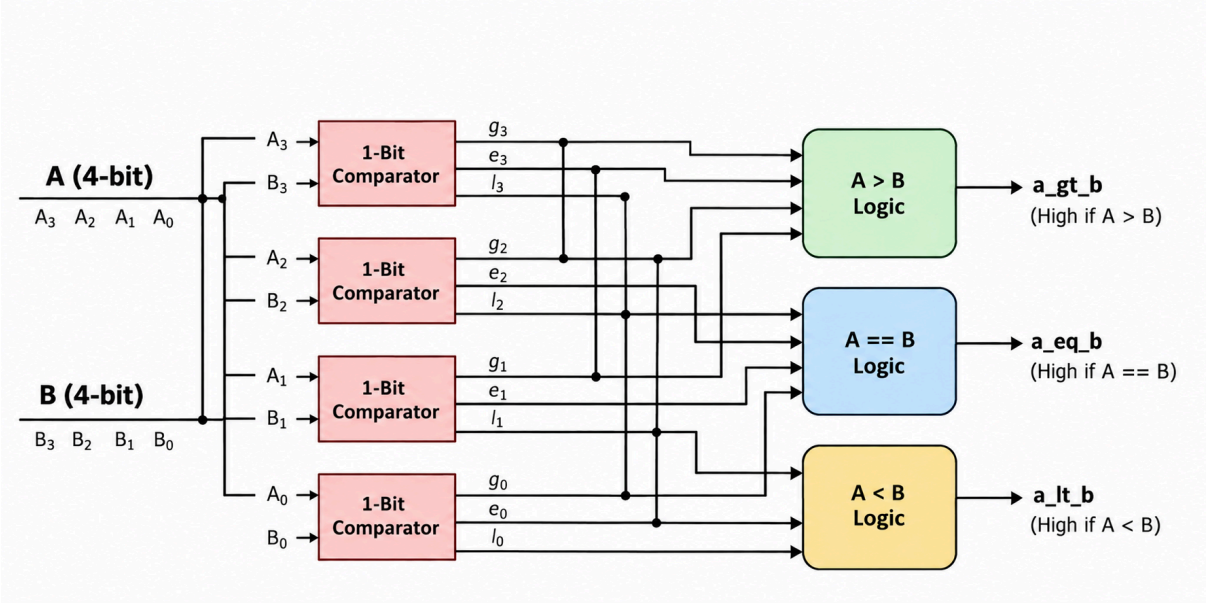
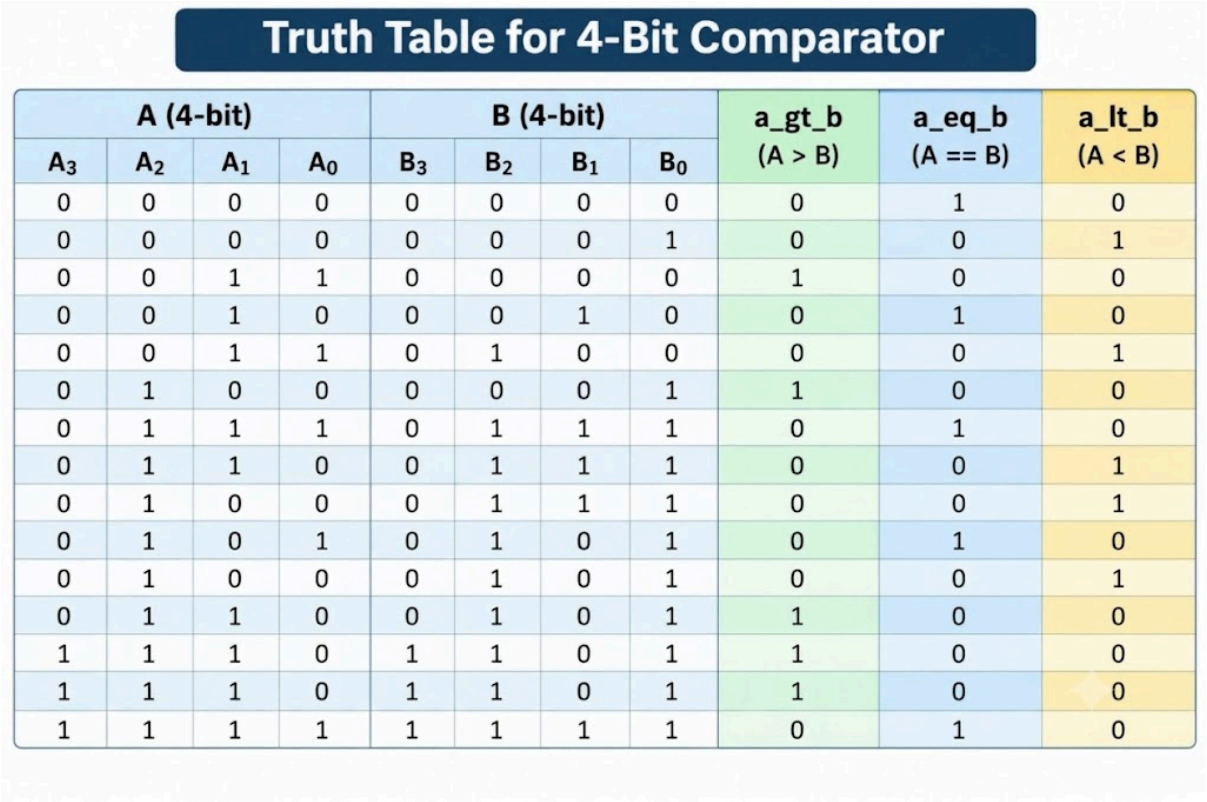


Figure 2.1: 4-Bit Comparator Block Diagram



A (4-bit)				B (4-bit)				a_gt_b (A > B)	a_eq_b (A == B)	a_lt_b (A < B)
A_3	A_2	A_1	A_0	B_3	B_2	B_1	B_0			
0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1	0	0	1
0	0	1	1	0	0	0	0	1	0	0
0	0	1	0	0	0	1	0	0	1	0
0	0	1	1	0	1	0	0	0	0	1
0	1	0	0	0	0	0	1	1	0	0
0	1	1	1	0	1	1	1	0	1	0
0	1	1	0	0	1	1	1	0	0	1
0	1	0	0	0	1	1	1	0	0	1
0	1	0	1	0	1	0	1	0	1	0
0	1	0	0	0	1	0	1	0	0	1
0	1	1	0	0	1	0	1	1	0	0
1	1	1	0	1	1	0	1	1	0	0
1	1	1	0	1	1	0	1	1	0	0
1	1	1	1	1	1	1	1	0	1	0

Table 2.1: Truth Table for 4-Bit Comparator

Verilog HDL Code

```
None
module comparator_4bit (input [3:0] a, b, output reg gt, eq, lt);
  always @(*) begin
    gt = (a > b); eq = (a == b); lt = (a < b);
  end
endmodule
```

Testbench

```
None
module tb_comparator_4bit;
  reg [3:0] a, b; wire gt, eq, lt;
  comparator_4bit uut (a, b, gt, eq, lt);
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_comparator_4bit);
    a = 4'b1000; b = 4'b0100; #10;
    a = 4'b0111; b = 4'b0111; #10;
    a = 4'b0010; b = 4'b1100; #10;
```

```
  end
endmodule
```

Output

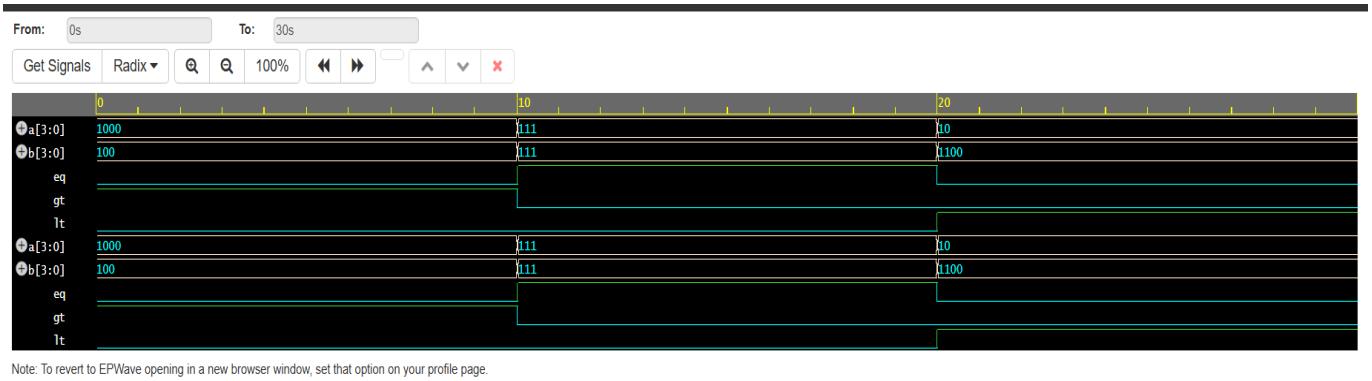


Figure 2.2: 4-Bit Comparator Simulation Waveform

3. 4-Bit Parity Generator and Checker

Design

- **Generator:** Uses XOR cascades to create an even parity bit. Generates `parity_bit` based on `d`.
- **Checker:** Evaluates a 5-bit input (4-bit data + 1-bit parity) to output `error_detected` if parity is odd.

Circuit Diagram & Truth Table

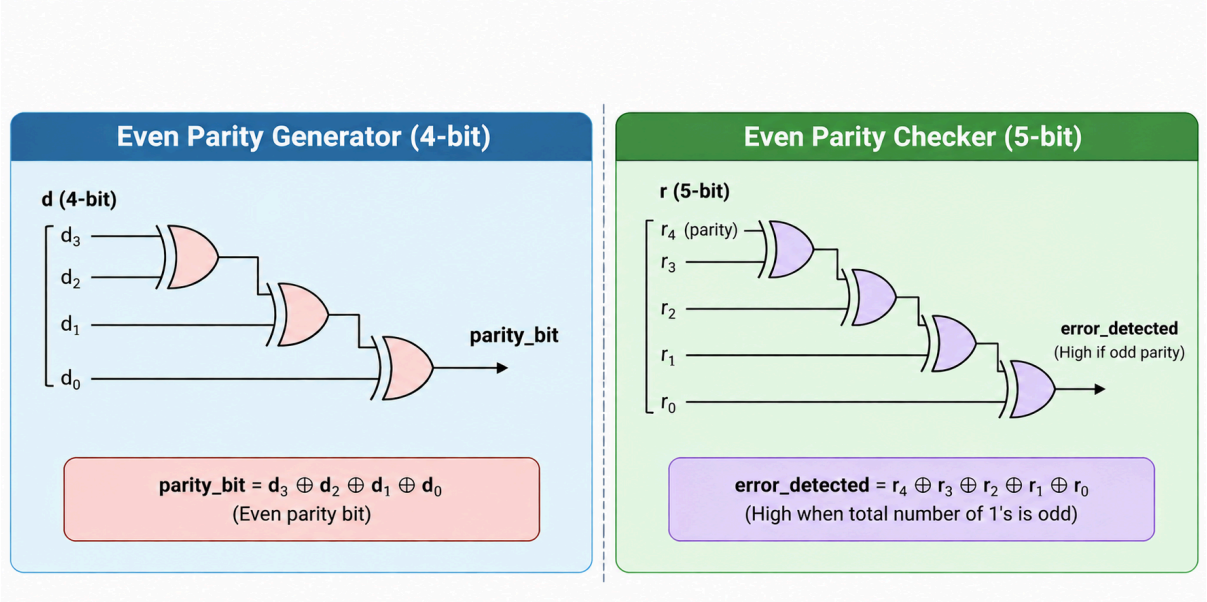


Figure 3.1: Even Parity Generator and Checker Block Diagrams

Truth Tables for Even Parity Generator and Checker (4-bit)

Even Parity Generator (4-bit)

Data Input (d)				Output
d_3	d_2	d_1	d_0	parity_bit
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Even Parity Checker (5-bit)

Received Input (r)					Output
r_4 (parity)	r_3	r_2	r_1	r_0	error_detected
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	1
0	0	0	1	1	0
0	0	1	0	0	1
0	0	1	0	1	0
0	0	1	1	0	0
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	0	1	0
0	1	0	1	0	0
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	1
1	0	1	0	0	0
1	0	1	0	1	1
1	0	1	1	0	1
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	1
1	1	0	1	0	1
1	1	0	1	1	0
1	1	1	0	0	1
1	1	1	0	1	0
1	1	1	1	0	1

Table 3.1: Truth Tables for Even Parity Generator and Checker

Verilog HDL Code

```
None
module parity_gen_chk (input [3:0] d, output p_bit, input [3:0] c_data, input
c_p, output err);
  assign p_bit = d[0] ^ d[1] ^ d[2] ^ d[3];
  assign err = c_data[0] ^ c_data[1] ^ c_data[2] ^ c_data[3] ^ c_p;
endmodule
```

Testbench

```
None
module tb_parity;
  reg [3:0] d, c_data; reg c_p; wire p_bit, err;
  parity_gen_chk uut (d, p_bit, c_data, c_p, err);
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_parity);
    d = 4'b1101; #10;
    d = 4'b1010; #10;
    c_data = 4'b1101; c_p = 1; #10;
    c_data = 4'b1101; c_p = 0; #10;
  end
endmodule
```

Output

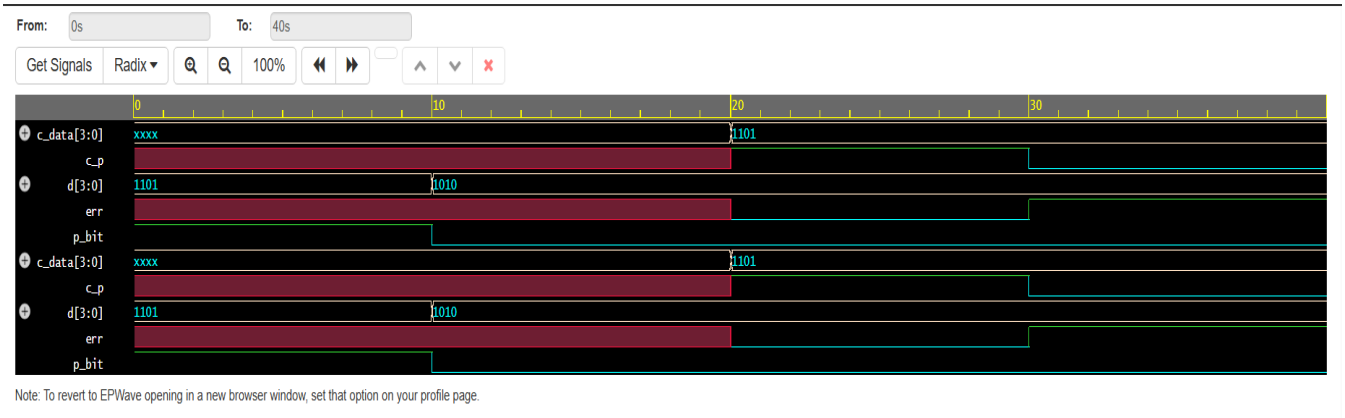


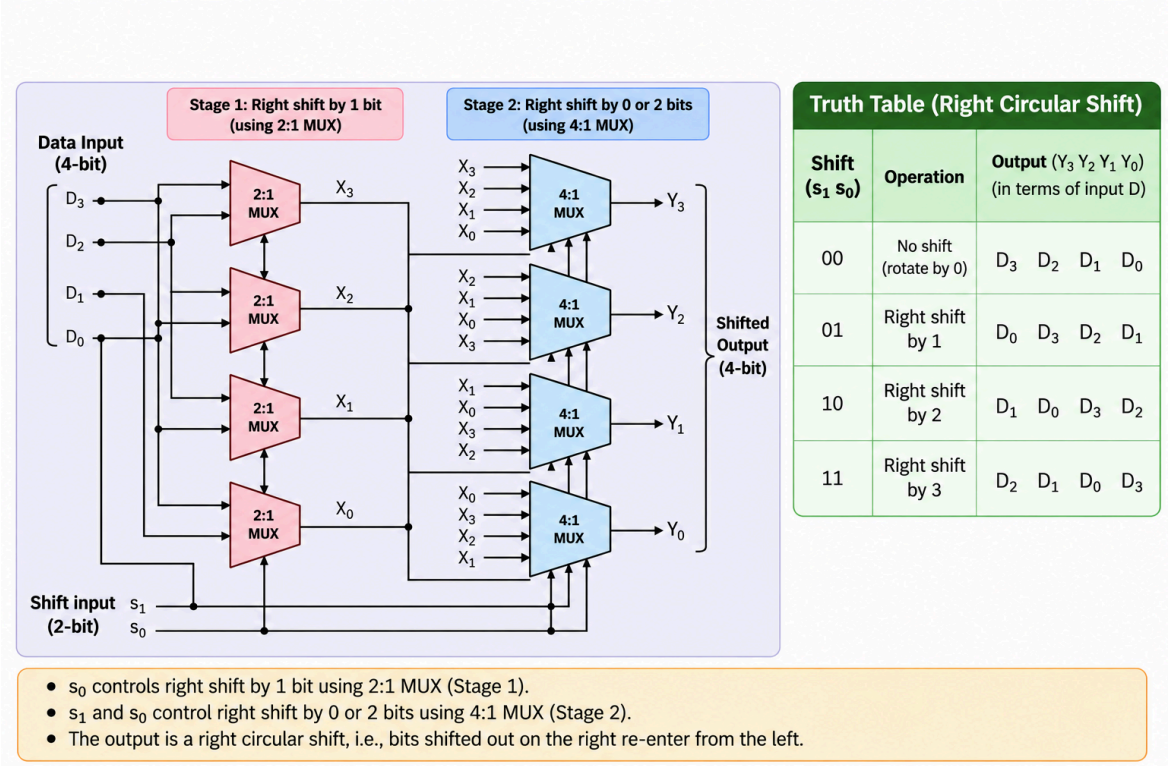
Figure 3.2: Parity Generator and Checker Simulation Waveform

4. 4-Bit Barrel Shifter

Design

This shifter cyclically rotates or shifts a 4-bit data stream to the right by `0` to `3` bit positions, specified by the `shift` input, using 2:1 and 4:1 multiplexers.

Circuit Diagram & Truth Table



Truth Table (Right Circular Shift)

Shift (s_1 s_0)	Operation	Output (Y_3 Y_2 Y_1 Y_0) (in terms of input D)
00	No shift (rotate by 0)	D_3 D_2 D_1 D_0
01	Right shift by 1	D_0 D_3 D_2 D_1
10	Right shift by 2	D_1 D_0 D_3 D_2
11	Right shift by 3	D_2 D_1 D_0 D_3

Figure 4.1: 4-Bit Barrel Shifter Logic Diagram and Truth Table

Verilog HDL Code

```
None
module barrel_shifter (input [3:0] din, input [1:0] shift, output reg [3:0]
dout);
  always @(*) begin
    case(shift)
      2'b00: dout = din;
      2'b01: dout = {din[0], din[3:1]};
      2'b10: dout = {din[1:0], din[3:2]};
      2'b11: dout = {din[2:0], din[3]};
    endcase
```

```
  end
endmodule
```

Testbench

```
None
module tb_barrel_shifter;
  reg [3:0] din; reg [1:0] shift; wire [3:0] dout;
  barrel_shifter uut (din, shift, dout);
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_barrel_shifter);
    din = 4'b1000;
    shift = 2'b00; #10;
    shift = 2'b01; #10;
    shift = 2'b10; #10;
    shift = 2'b11; #10;
  end
endmodule
```

Output

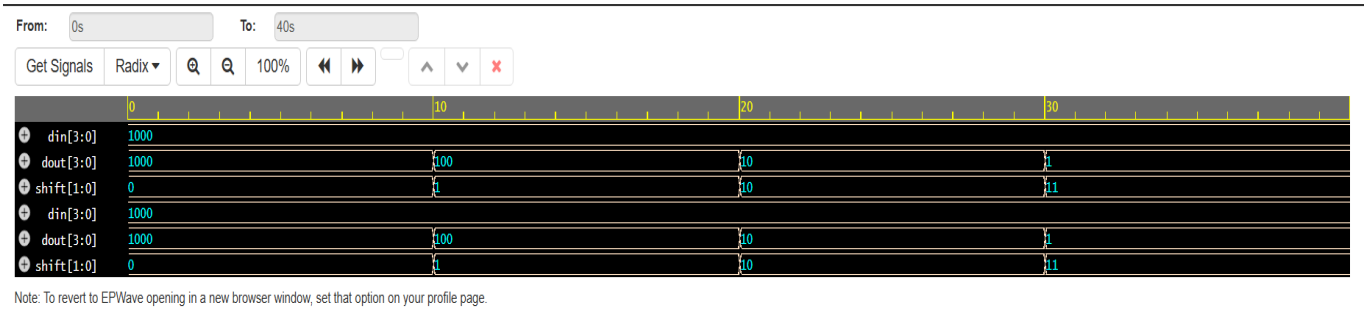


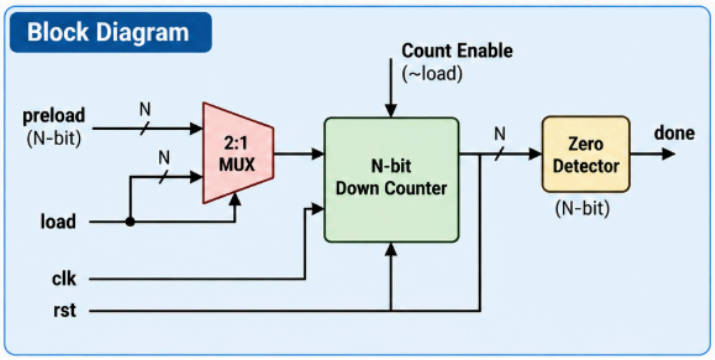
Figure 4.2: 4-Bit Barrel Shifter Simulation Waveform

5. Timer Module

Design

A timer down-counts from a loaded preload value to 0 and asserts a `done` signal when 0 is reached. Useful for timing events over `CLK` cycles.

Circuit Diagram & Truth Table



Operation Table

rst	load	Operation (on rising edge of clk)	done
1	X	Counter cleared to 0	1
0	1	Load preload value count ← preload	1 if preload = 0
0	0	Count down count ← count − 1	1 when count = 0

Figure 5.1: Timer Module Block Diagram and Operation Table

Verilog HDL Code

```
None
module timer (input clk, rst, load, input [3:0] preload, output reg done);
  reg [3:0] count;
  always @(posedge clk or posedge rst) begin
    if (rst) {count, done} <= 5'b00000;
    else if (load) {count, done} <= {preload, 1'b0};
    else if (count > 0) begin count <= count - 1; done <= (count == 1); end
    else done <= 1'b1;
  end
endmodule
```

Testbench

```
None
module tb_timer;
  reg clk=0, rst=1, load=0; reg [3:0] preload=0; wire done;
  timer uut (clk, rst, load, preload, done);
  always #5 clk = ~clk;
```

```
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_timer);
    #10 rst = 0; load = 1; preload = 4'd4;
    #10 load = 0;
    #60 $finish;
  end
endmodule
```

Output

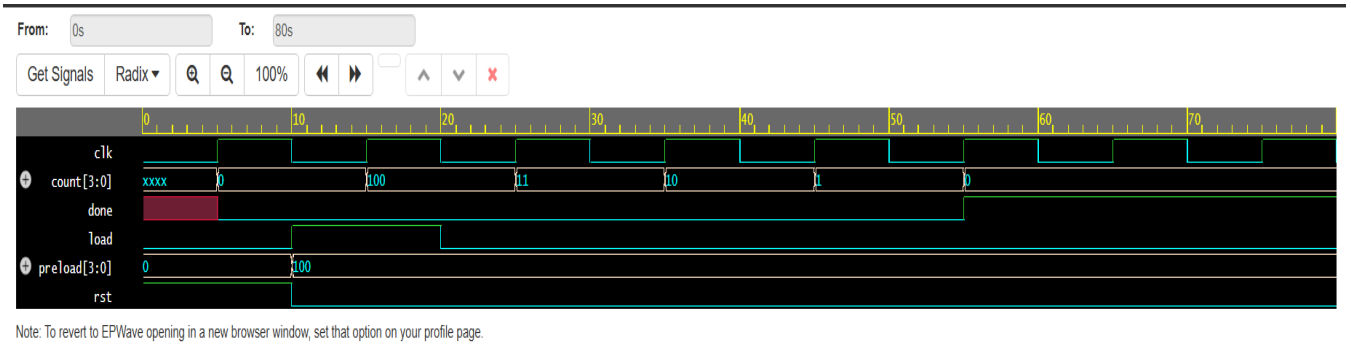


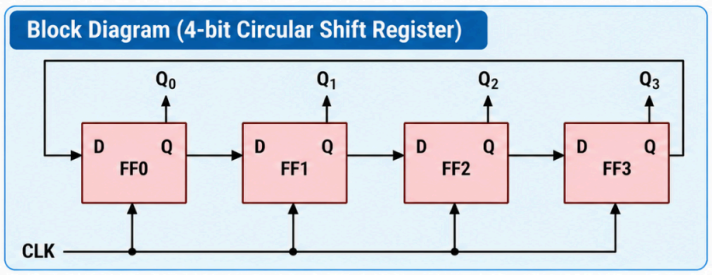
Figure 5.2: Timer Module Simulation Waveform

6. Ring Counter

Design

A sequential shift register where the last flip-flop feeds back into the first. This recirculates a single `'1'` (or `'0'`) pattern in a loop.

Circuit Diagram & Truth Table



State Table / Sequence (Single '1' Circulating)

Clock Cycle	Outputs (Q_3 Q_2 Q_1 Q_0)			
Initial	0	0	0	1
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1
...	(repeats)			

Figure 6.1: Ring Counter Block Diagram and State Sequence Table

Verilog HDL Code

```
None
module ring_counter (input clk, rst, output reg [3:0] q);
  always @(posedge clk or posedge rst)
    if (rst) q <= 4'b0001;
    else q <= {q[0], q[3:1]};
endmodule
```

Testbench

```
None
module tb_ring;
  reg clk=0, rst=1; wire [3:0] q;
  ring_counter uut (clk, rst, q);
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_ring);
    #10 rst = 0; #50 $finish;
  end
endmodule
```

Output

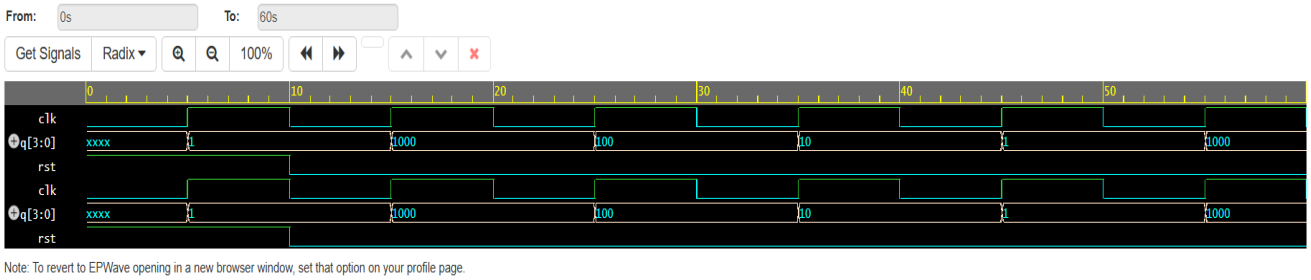


Figure 6.2: Ring Counter Simulation Waveform

7. Johnson Counter

Design

A sequential shift register where the inverted output of the last flip-flop is fed back as the input to the first. Results in a count length of 2N.

Circuit Diagram & Truth Table

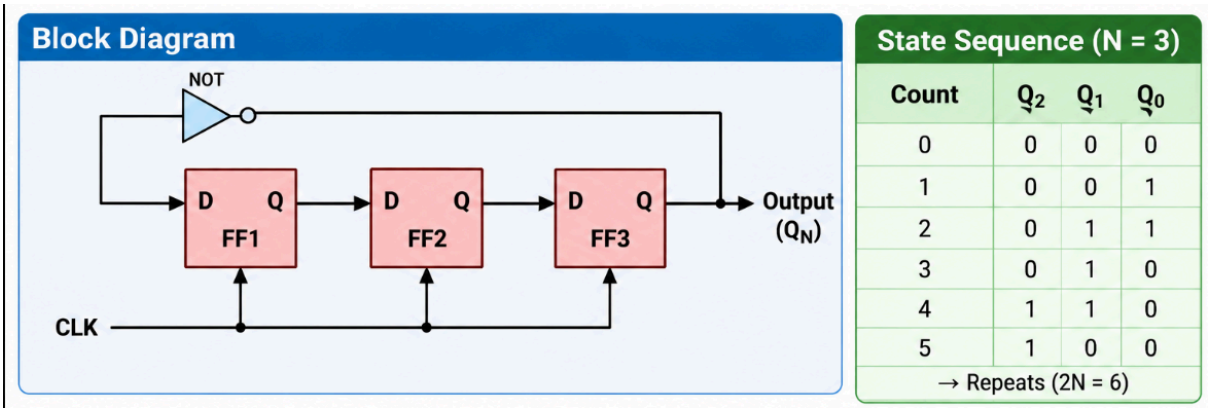


Figure 7.1: Johnson Counter Block Diagram and State Sequence Table

Verilog HDL Code

```
None
module johnson_counter (input clk, rst, output reg [3:0] q);
  always @(posedge clk or posedge rst)
    if (rst) q <= 4'b0000;
    else q <= {~q[0], q[3:1]};
endmodule
```

Testbench

```
None
module tb_johnson;
  reg clk=0, rst=1; wire [3:0] q;
  johnson_counter uut (clk, rst, q);
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_johnson);
```

```
    #10 rst = 0; #80 $finish;
  end
endmodule
```

Output

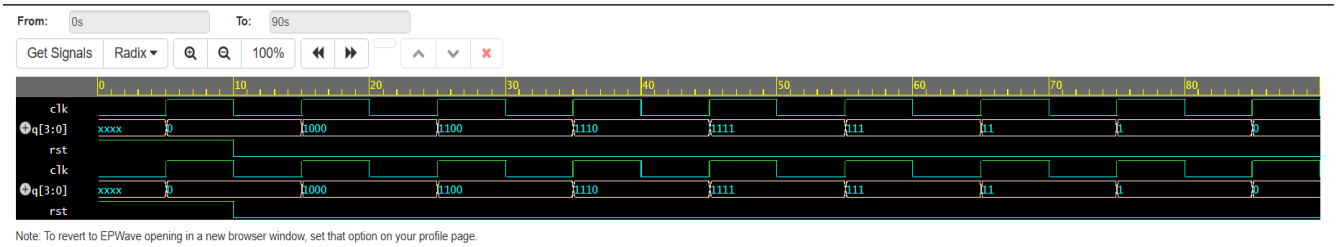


Figure 7.2: Johnson Counter Simulation Waveform

8. Synchronous and Asynchronous Counters

Design:

- **Synchronous:** All flip-flops transition simultaneously on the active clock edge, using combination lookahead logic.
- **Asynchronous (Ripple):** Each flip-flop is clocked by the output of the previous flip-flop, creating cascading ripple propagation.

Circuit Diagram & Truth Table

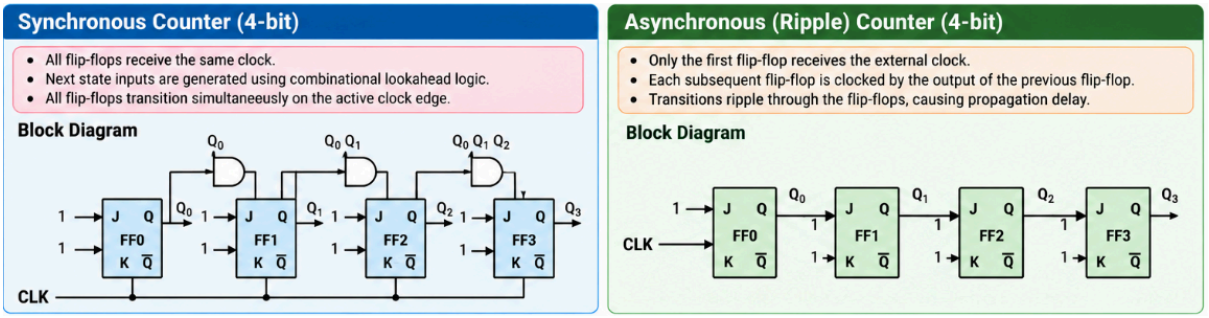


Figure 8.1: Synchronous and Asynchronous Counters Block Diagrams

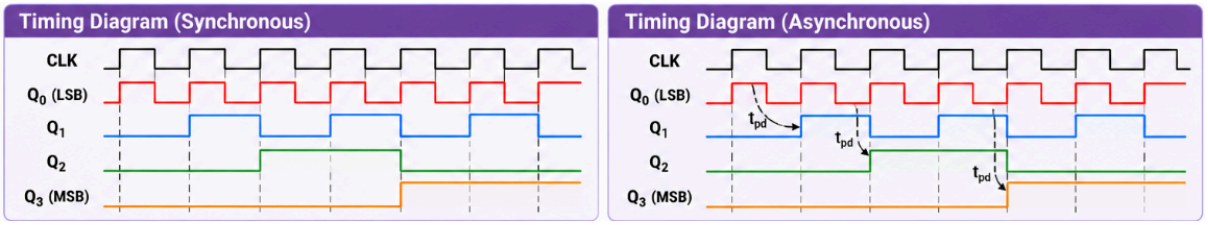


Figure 8.2: Synchronous vs. Asynchronous Counters Timing Diagram

Verilog HDL Code

```
None
module sync_async_counters (
  input clk,
  input rst,
  output reg [3:0] q_sync,
  output reg [3:0] q_async
);
  // Synchronous Counter
  always @(posedge clk or posedge rst) begin
    if (rst)
      q_sync <= 4'b0000;
```

```
    else
      q_sync <= q_sync + 1;
  end
  // Asynchronous Ripple Counter Model
  always @(posedge clk or posedge rst) begin
    if (rst) begin
      q_async[0] <= 1'b0;
    end else begin
      q_async[0] <= ~q_async[0];
    end
  end
  always @(negedge q_async[0] or posedge rst) begin
    if (rst) q_async[1] <= 1'b0;
    else     q_async[1] <= ~q_async[1];
  end
  always @(negedge q_async[1] or posedge rst) begin
    if (rst) q_async[2] <= 1'b0;
    else     q_async[2] <= ~q_async[2];
  end
  always @(negedge q_async[2] or posedge rst) begin
    if (rst) q_async[3] <= 1'b0;
    else     q_async[3] <= ~q_async[3];
  end
endmodule
```

Testbench

None

```
module tb_counters;
  reg clk, rst;
  wire [3:0] q_sync, q_async;
  sync_async_counters uut (.clk(clk), .rst(rst), .q_sync(q_sync),
.q_async(q_async));
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_counters);
    $monitor("Time=%0t | rst=%b | Sync=%d Async=%d", $time, rst, q_sync,
q_async);
    clk = 0; rst = 1; #12 rst = 0;
    #180 $finish;
  end
endmodule
```

Output

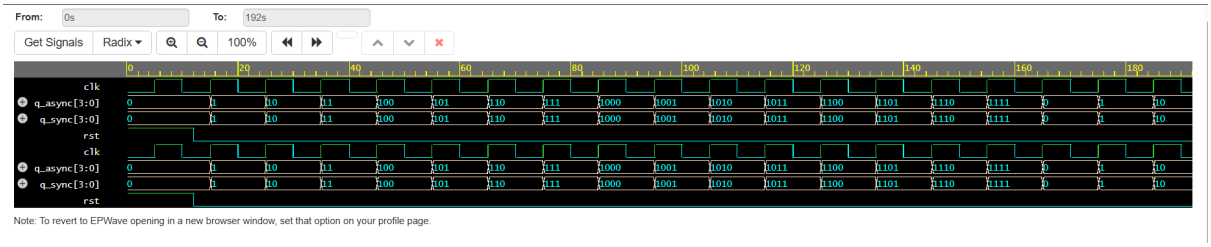


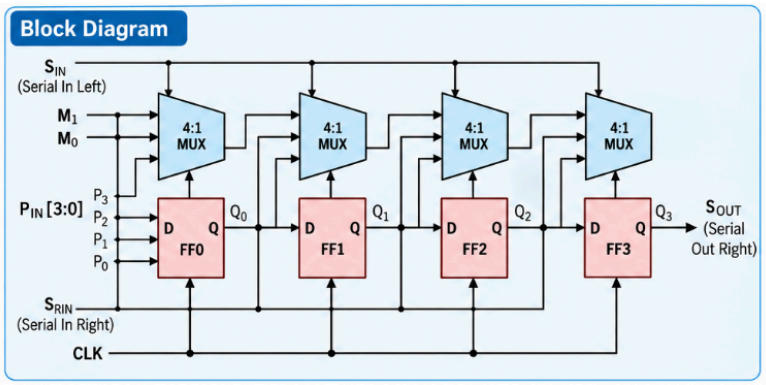
Figure 8.3: Synchronous and Asynchronous Counters Simulation Waveform

9. Shift Register

Design

A 4-bit Universal Shift Register that handles multiple serial/parallel loading operations depending on the `mode` control bits.

Circuit Diagram & Truth Table



Mode Selection and Operation

M1	M_0	Operation	Next State (Q_3 Q_2 Q_1 Q_0)
0	0	Hold (No change)	Q_3 Q_2 Q_1 Q_0
0	1	Shift Right (Serial In Right)	Q_0 Q_3 Q_2 Q_1
1	0	Shift Left (Serial In Left)	Q_2 Q_1 Q_0 S_{IN}
1	1	**Parallel Load**	P_3 P_2 P_1 P_0

Figure 9.1: Universal Shift Register Block Diagram and Operation Table

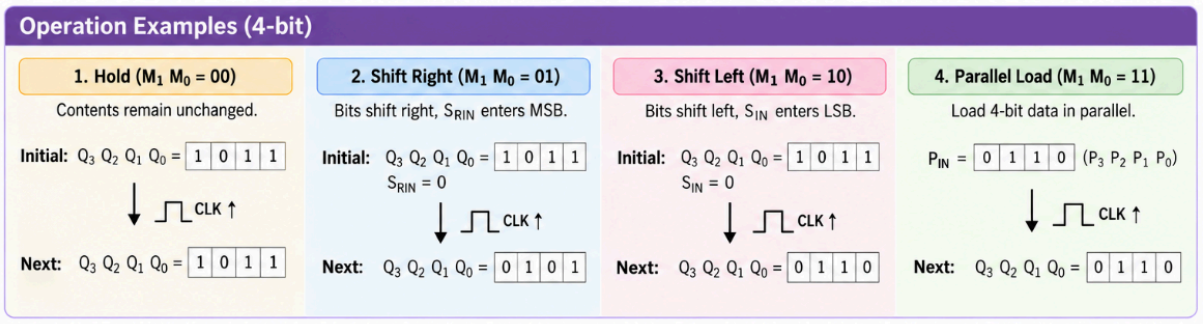


Figure 9.2: Shift Register Operation Mode Examples

Verilog HDL Code

```
None
module shift_reg (input clk, rst, sin, input [1:0] mode, input [3:0] din,
output reg [3:0] dout);
  always @(posedge clk or posedge rst) begin
    if (rst) dout <= 0;
    else case(mode)
      2'b01: dout <= {sin, dout[3:1]};
      2'b10: dout <= {dout[2:0], sin};
      2'b11: dout <= din;
```

```
    endcase
  end
endmodule
```

Testbench

None

```
module tb_shift;
  reg clk=0, rst=1, sin=0; reg [1:0] mode=0; reg [3:0] din=0; wire [3:0] dout;
  shift_reg uut (clk, rst, sin, mode, din, dout);
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_shift);
    #10 rst = 0; mode = 2'b11; din = 4'b1011;
    #10 mode = 2'b01; sin = 1;
    #10 sin = 0;
    #20 $finish;
  end
endmodule
```

Output

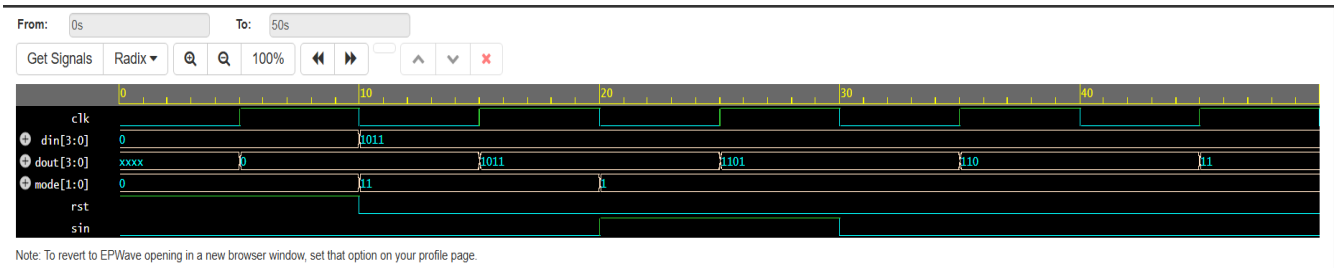


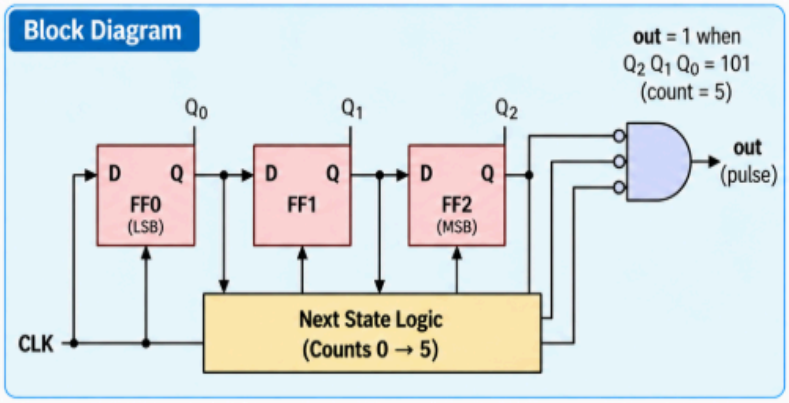
Figure 9.3: Universal Shift Register Simulation Waveform

10. Divide-by-N Counter (Using Modulo-6)

Design

Divides the input clock frequency by N (here N = 6, counting 0 to 5). The output out pulse has an active-high assertion every sixth count tick.

Circuit Diagram & Truth Table



State Table (Counts 0 to 5)

Decimal Count	Q_2	Q_1	Q_0	out
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	0
5	1	0	1	1

Figure 10.1: Divide-by-6 Counter Block Diagram and State Table

Verilog HDL Code

```
None
module div_n (input clk, rst, output reg clk_out);
  reg [2:0] count;
  always @(posedge clk or posedge rst) begin
    if (rst) {count, clk_out} <= 0;
    else if (count == 5) begin count <= 0; clk_out <= 1; end
    else begin count <= count + 1; clk_out <= 0; end
  end
endmodule
```

Testbench

```
None
module tb_div_n;
  reg clk=0, rst=1; wire clk_out;
  div_n uut (clk, rst, clk_out);
```

```
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_div_n);
    #10 rst = 0; #100 $finish; end
endmodule
```

Output

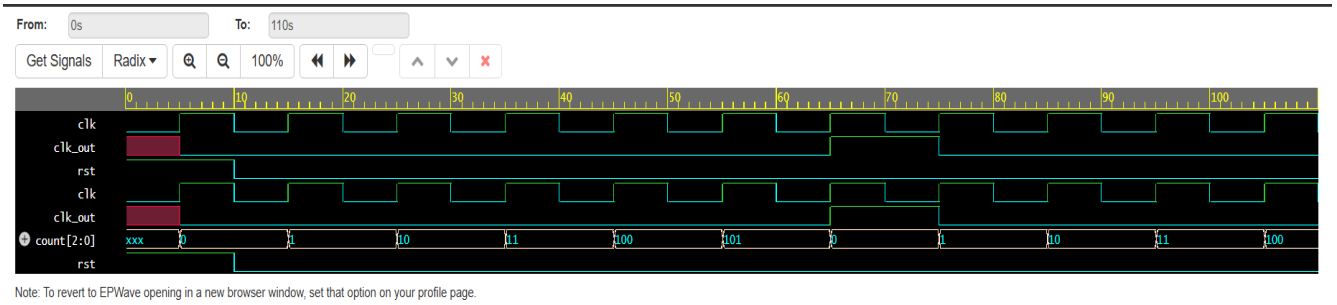


Figure 10.2: Divide-by-6 Counter Simulation Waveform

11. Mealy and Moore FSMs

---

a. Mealy Finite State Machine

Design (Mealy Machine)

Detects a "10" pattern. The output z is dependent on both the current state and the input (x).

Circuit Diagram & Truth Table

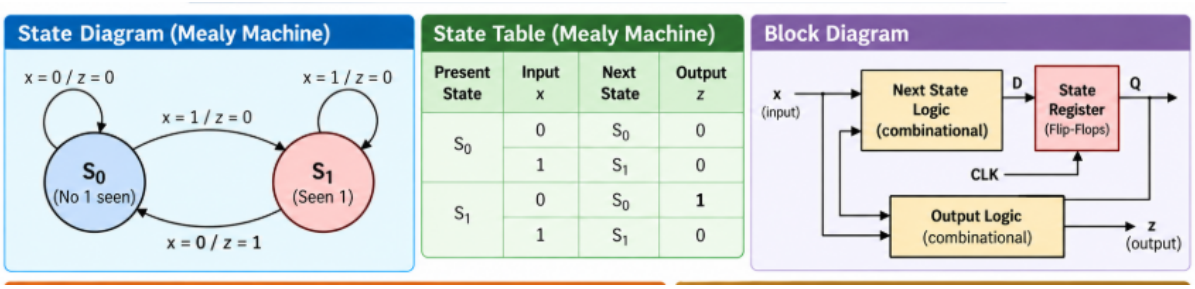


Figure 11.1: Mealy FSM State Diagram, State Table, and Block Diagram

Verilog HDL Code

```
None
module mealy_fsm (
  input clk, rst, x,
  output reg z
);
  reg state, next_state;
  parameter S0 = 1'b0, S1 = 1'b1;
  always @(posedge clk or posedge rst) begin
    if (rst) state <= S0;
    else     state <= next_state;
  end
  always @(*) begin
    z = 1'b0;
    case (state)
      S0: begin
        next_state = x ? S1 : S0;
      end
      S1: begin
        next_state = x ? S1 : S0;
        z = ~x; // Output changes on inputs (high when input is 0)
      end
    endcase
```

```
  end
endmodule
```

Testbench

None

```
module tb_mealy_fsm;
  reg clk, rst, x;
  wire z;
  mealy_fsm uut (.clk(clk), .rst(rst), .x(x), .z(z));
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_mealy_fsm);
    $monitor("Time=%0t | rst=%b x=%b | State=%b z=%b", $time, rst, x,
uut.state, z);
    clk = 0; rst = 1; x = 0;
    #12 rst = 0;
    #10 x = 1; // Transition to S1
    #10 x = 0; // "10" input, z=1
    #10 x = 0;
    #10 x = 1;
    #10 x = 0; // "10" input, z=1
    #10 $finish;
  end
endmodule
```

Output

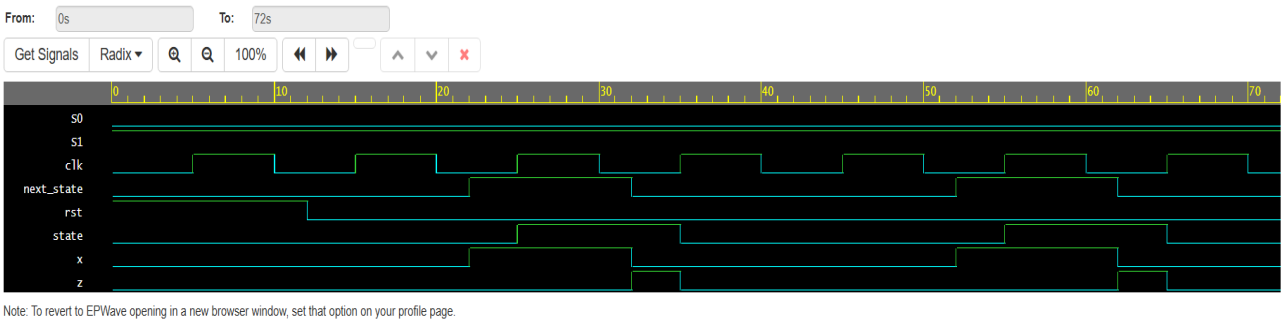


Figure 11.2: Mealy FSM Simulation Waveform

b. Moore Finite State Machine

Design (Moore Machine)

The output z is dependent solely on current state. Transition through states happens based on x to detect pattern "11".

Circuit Diagram & Truth Table

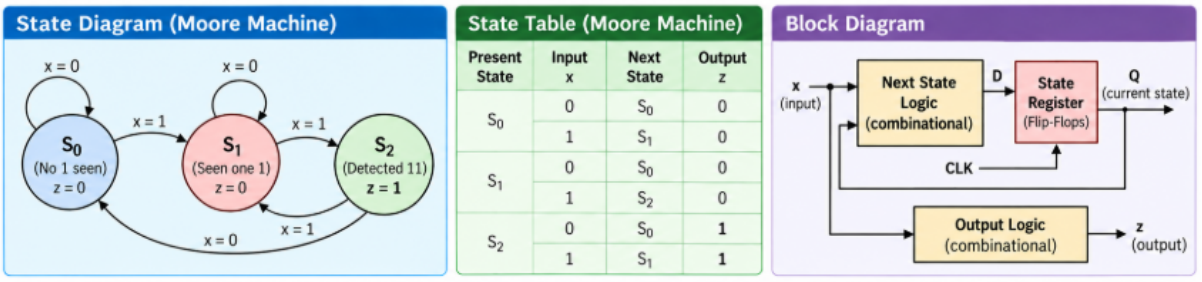


Figure 11.3: Moore FSM State Diagram, State Table, and Block Diagram

Verilog HDL Code

```
None
module moore_fsm (
  input clk, rst, x,
  output reg z
);
  reg [1:0] state, next_state;
  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
  always @(posedge clk or posedge rst) begin
    if (rst) state <= S0;
    else     state <= next_state;
  end
  always @(*) begin
    case(state)
      S0: next_state = x ? S1 : S0;
      S1: next_state = x ? S2 : S0;
      S2: next_state = x ? S2 : S0;
      default: next_state = S0;
    endcase
  end
  always @(*) begin
    z = (state == S2); // Only relies on the state
  end
endmodule
```

Testbench

```
None
module tb_moore_fsm;
  reg clk, rst, x;
  wire z;
  moore_fsm uut (.clk(clk), .rst(rst), .x(x), .z(z));
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_moore_fsm);
    $monitor("Time=%0t | rst=%b x=%b | State=%b z=%b", $time, rst, x,
uut.state, z);
    clk = 0; rst = 1; x = 0;
    #12 rst = 0;
    #10 x = 1;
    #10 x = 1; // Transitions to S2 (pattern detected, z=1)
    #10 x = 0; // State drops back
    #20 $finish;
  end
endmodule
```

Output

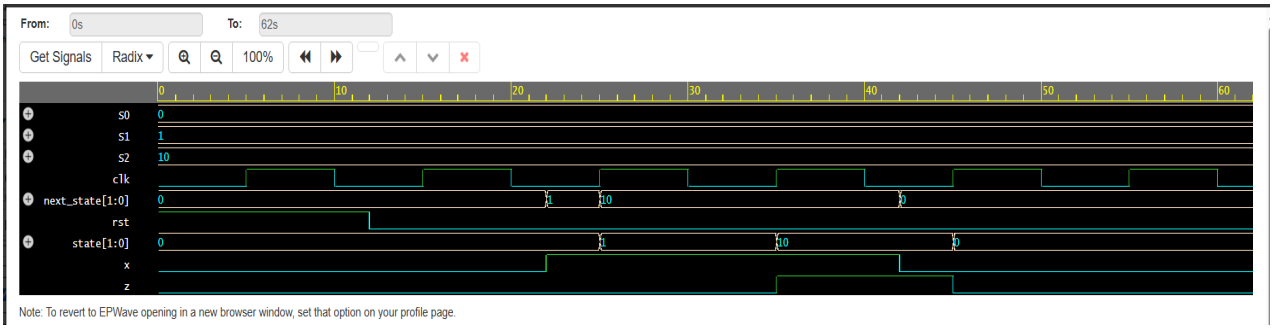


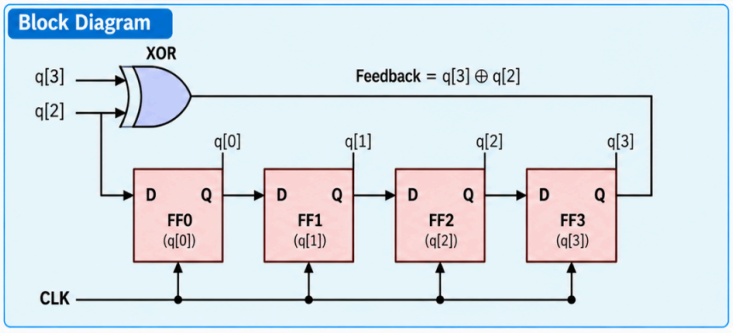
Figure 11.4: Moore FSM Simulation Waveform

12. Random Sequence Generator (LFSR)

Design

The Pseudo-Random Binary Sequence generator is designed using a 4-bit Linear Feedback Shift Register (LFSR) with an exclusive-OR tap feedback path. We tap out of `q[3]` and `q[2]` (`N = 4`, xor tapped).

Circuit Diagram & Truth Table



State Table (N = 4, taps: q[3] and q[2])

Current State				Feedback	Next State			
q[3]	q[2]	q[1]	q[0]	q[3] ⊕ q[2]	q[3]	q[2]	q[1]	q[0]
0	0	0	1	0	0	0	1	0
0	0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0	0
1	0	0	0	1	0	1	0	0
0	1	0	1	0	0	1	0	1
0	1	0	1	0	1	0	1	0
1	0	1	0	1	0	1	0	1
0	1	1	1	0	1	0	1	1
1	0	1	1	1	1	1	1	0
1	1	1	0	0	1	1	1	1
1	1	1	1	0	0	1	1	1
0	1	1	1	0	1	0	1	1
0	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1

Figure 12.1: 4-Bit LFSR Block Diagram and State Table

Verilog HDL Code

```
None
module lfsr_4bit (
  input clk,
  input rst,
  output reg [3:0] q
);
  wire feedback;
  assign feedback = q[3] ^ q[2]; // feedback tapping for 4-bit sequence
  always @(posedge clk or posedge rst) begin
    if (rst) begin
      q <= 4'b1000; // Seed value (Must be non-zero to jump-start)
    end else begin
      q <= {q[2:0], feedback};
    end
  end
endmodule
```

Testbench

```
None
module tb_lfsr_4bit;
  reg clk, rst;
  wire [3:0] q;
  lfsr_4bit uut (.clk(clk), .rst(rst), .q(q));
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_lfsr_4bit);
    $monitor("Time=%0t | rst=%b | Random State Sequence=%b (%d)", $time, rst,
q, q);
    clk = 0; rst = 1; #12 rst = 0;
    #150 $finish;
  end
endmodule
```

Output

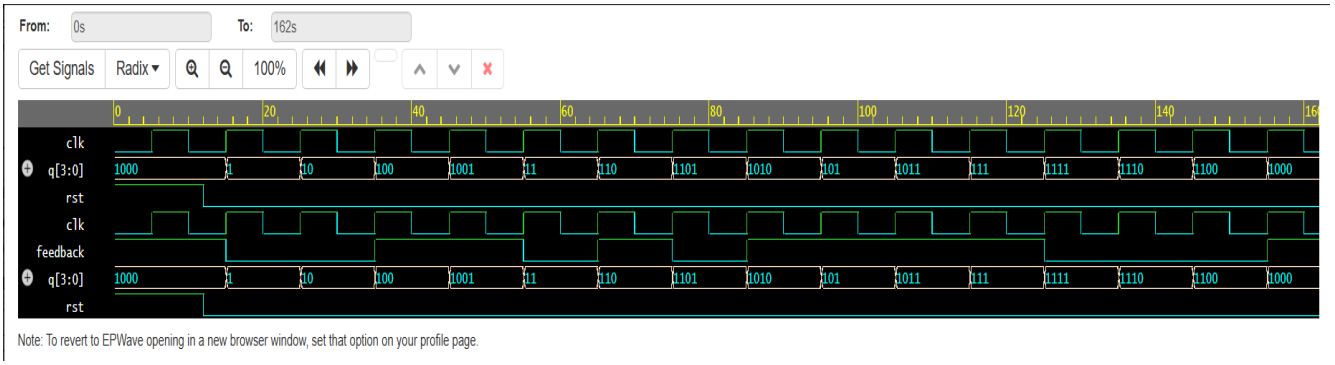


Figure 12.2: 4-Bit LFSR Simulation Waveform